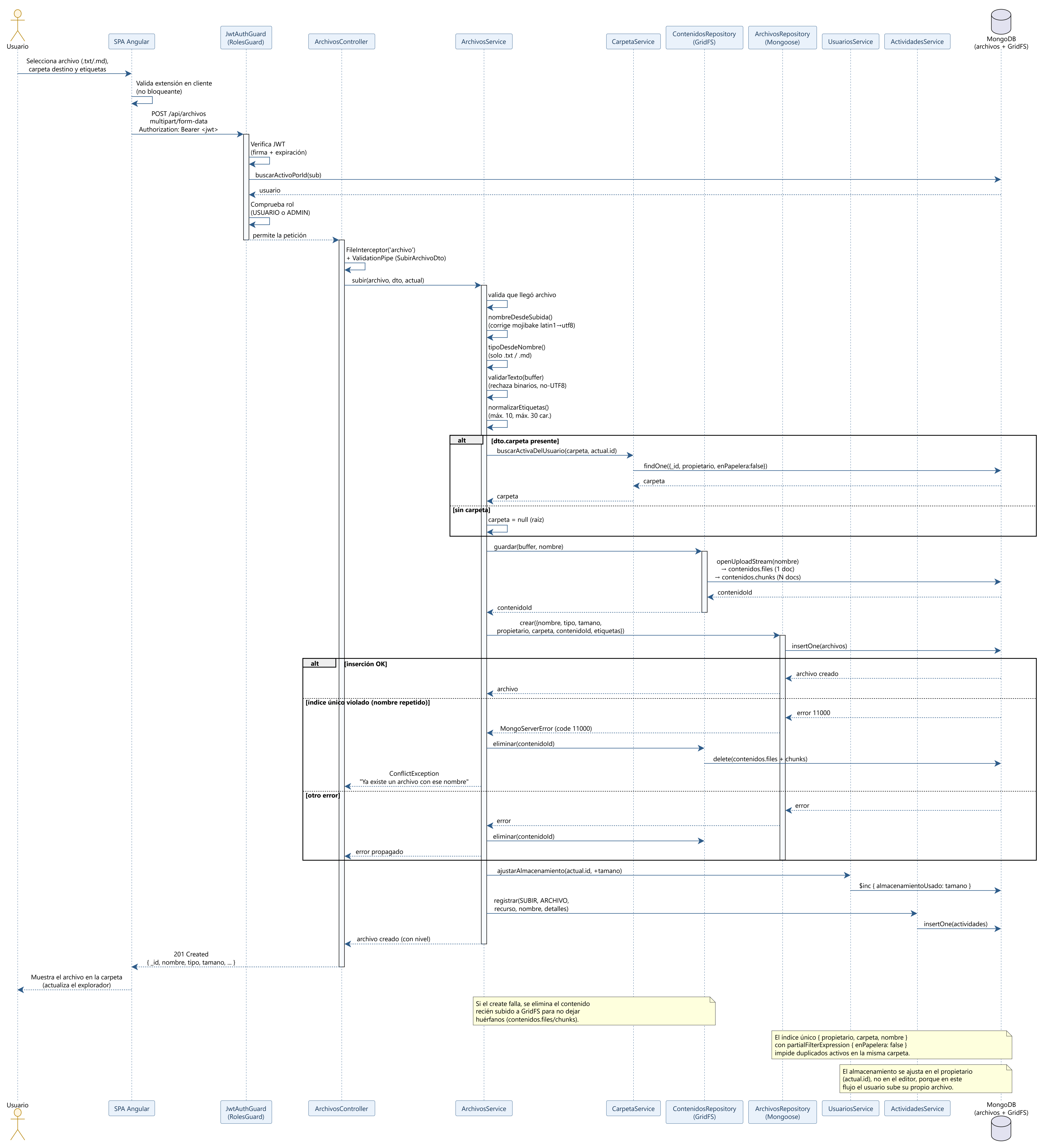
Carga y descarga

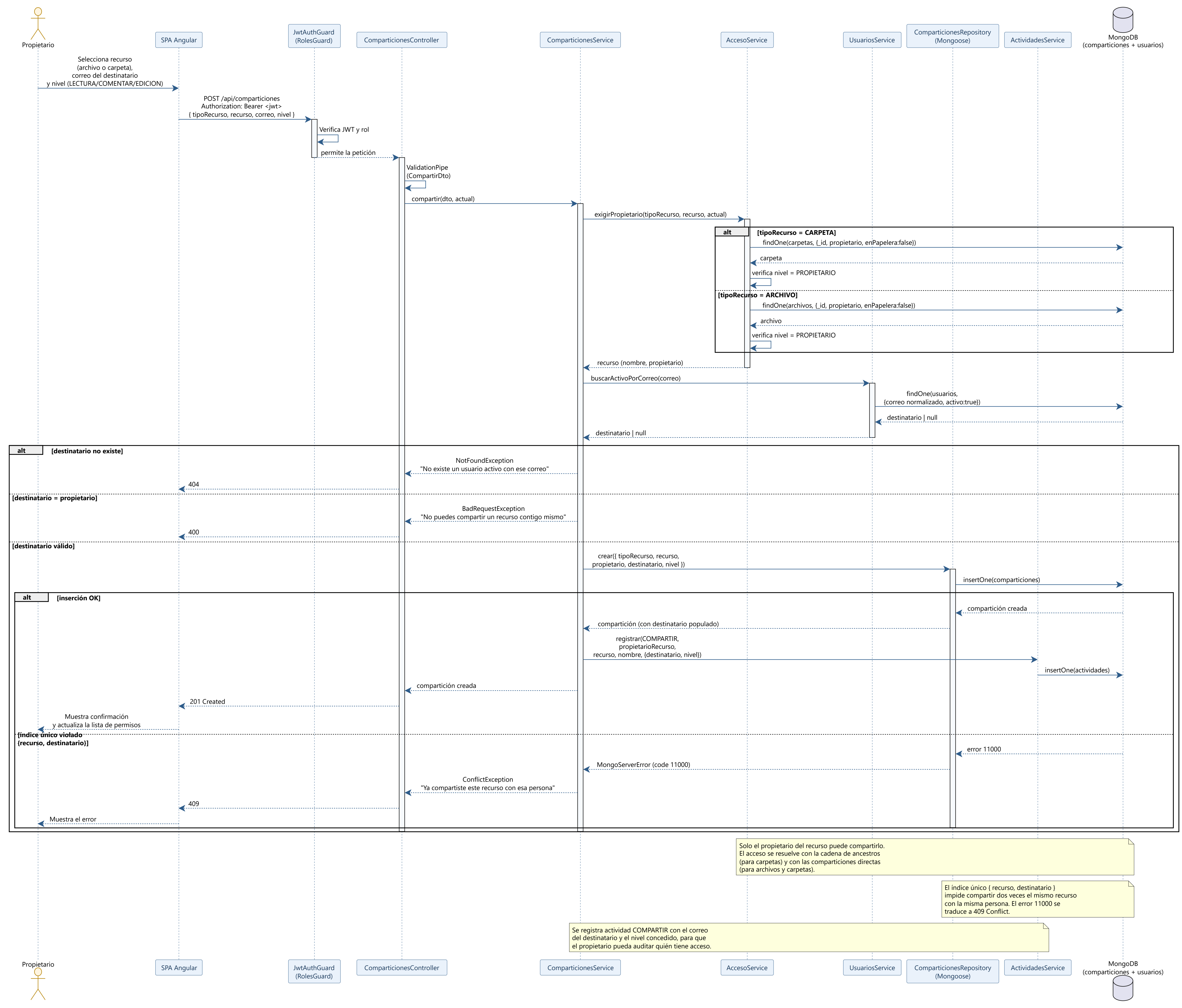

Usuario
SPA Angular
JwtAuthGuard (RolesGuard)
ArchivosController
ArchivosService
CarpetaService
ContenidosRepository (GridFS)
ArchivosRepository (Mongoose)
UsuariosService
ActividadesService
MongoDB (archivos + GridFS)
Selecciona archivo (.txt/.md), carpeta destino y etiquetas
Valida extensión en cliente (no bloqueante)
POST /api/archivos multipart/form-data Authorization: Bearer <jwt>
Verifica JWT (firma + expiración)
buscarActivoPorId(sub)
usuario
Comprueba rol (USUARIO o ADMIN)
permite la petición
FileInterceptor('archivo') + ValidationPipe (SubirArchivoDto)
subir(archivo, dto, actual)
valida que llegó archivo
nombreDesdeSubida() (corrige mojibake latin1→utf8)
tipoDesdeNombre() (solo .txt / .md)
validarTexto(buffer) (rechaza binarios, no-UTF8)
normalizarEtiquetas() (máx. 10, máx. 30 car.)
alt
[dto.carpeta presente]
buscarActivaDelUsuario(carpeta, actual.id)
findOne({_id, propietario, enPapelera:false})
carpeta
carpeta
[sin carpeta]
carpeta = null (raíz)
guardar(buffer, nombre)
openUploadStream(nombre) → contenidos.files (1 doc) → contenidos.chunks (N docs)
contenidoId
contenidoId
crear({nombre, tipo, tamano, propietario, carpeta, contenidoId, etiquetas})
insertOne(archivos)
alt
[inserción OK]
archivo creado
archivo
[índice único violado (nombre repetido)]
error 11000
MongoServerError (code 11000)
eliminar(contenidoId)
delete(contenidos.files + chunks)
ConflictException "Ya existe un archivo con ese nombre"
[otro error]
error
error
eliminar(contenidoId)
error propagado
ajustarAlmacenamiento(actual.id, +tamano)
$inc { almacenamientoUsado: tamano }
registrar(SUBIR, ARCHIVO, recurso, nombre, detalles)
insertOne(actividades)
archivo creado (con nivel)
201 Created { _id, nombre, tipo, tamano, ... }
Muestra el archivo en la carpeta (actualiza el explorador)

Si el create falla, se elimina el contenido recién subido a GridFS para no dejar huérfanos (contenidos.files/chunks).

El índice único { propietario, carpeta, nombre } con partialFilterExpression { enPapelera: false } impide duplicados activos en la misma carpeta.

El almacenamiento se ajusta en el propietario (actual.id), no en el editor, porque en este flujo el usuario sube su propio archivo.

Permisos

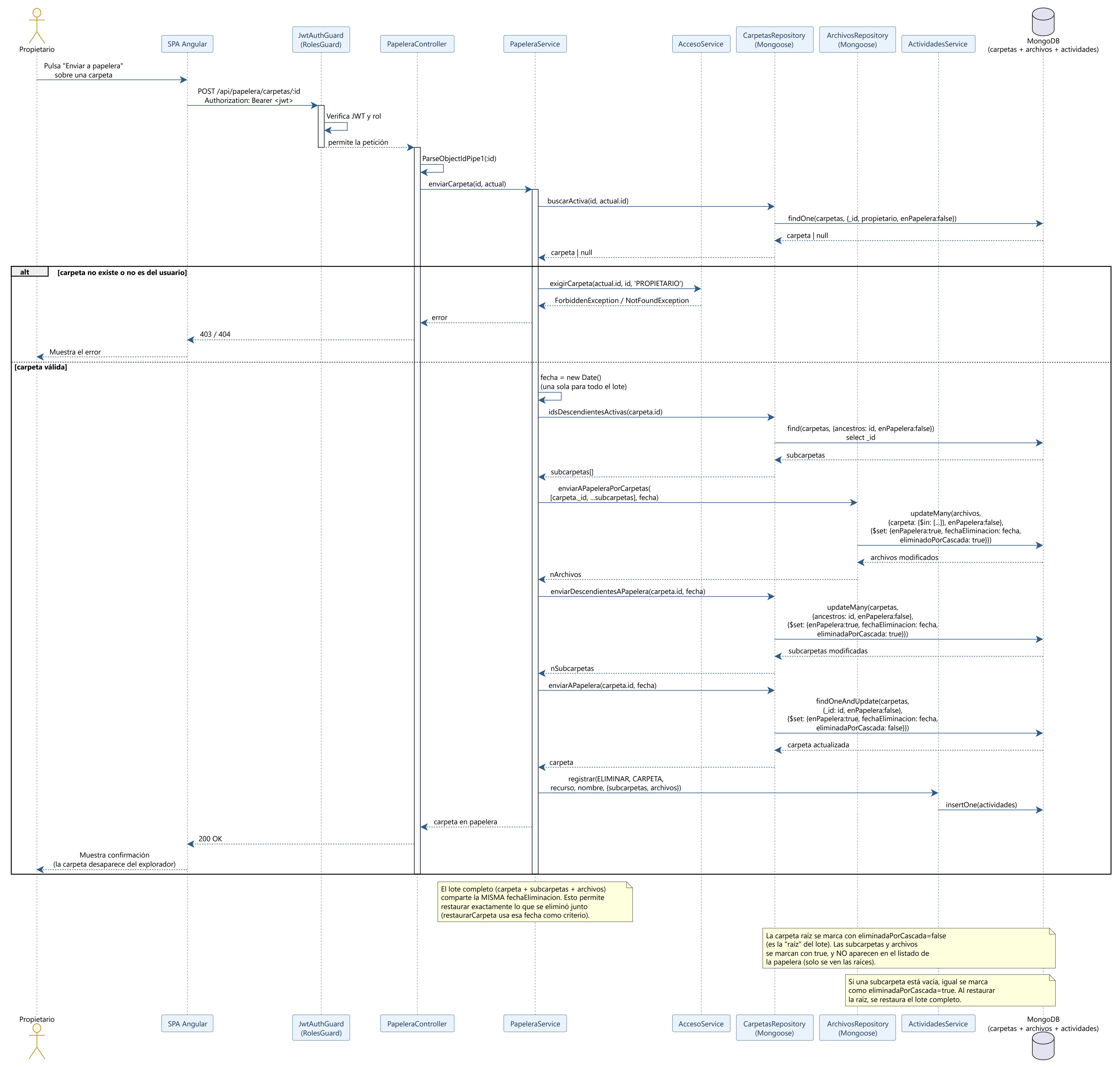

Propietario
SPA Angular
JwtAuthGuard (RolesGuard)
ComparticionesController
ComparticionesService
AccesoService
UsuariosService
ComparticionesRepository (Mongoose)
ActividadesService
MongoDB (comparticiones + usuarios)
Selecciona recurso (archivo o carpeta), correo del destinatario y nivel (LECTURA/COMENTAR/EDICION)
POST /api/comparticiones Authorization: Bearer <jwt> { tipoRecurso, recurso, correo, nivel }
Verifica JWT y rol
permite la petición
ValidationPipe (CompartirDto)
compartir(dto, actual)
exigirPropietario(tipoRecurso, recurso, actual)
alt
[tipoRecurso = CARPETA]
findOne(carpetas, {_id, propietario, enPapelera:false})
carpeta
verifica nivel = PROPIETARIO
[tipoRecurso = ARCHIVO]
findOne(archivos, {_id, propietario, enPapelera:false})
archivo
verifica nivel = PROPIETARIO
recurso (nombre, propietario)
buscarActivoPorCorreo(correo)
findOne(usuarios, {correo normalizado, activo:true})
destinatario | null
destinatario | null
alt
[destinatario no existe]
NotFoundException "No existe un usuario activo con ese correo"
404
[destinatario = propietario]
BadRequestException "No puedes compartir un recurso contigo mismo"
400
[destinatario válido]
crear({ tipoRecurso, recurso, propietario, destinatario, nivel })
insertOne(comparticiones)
alt
[inserción OK]
compartición creada
compartición (con destinatario populado)
registrar(COMPARTIR, propietarioRecurso, recurso, nombre, {destinatario, nivel})
insertOne(actividades)
compartición creada
201 Created
Muestra confirmación y actualiza la lista de permisos
[índice único violado {recurso, destinatario}]
error 11000
MongoServerError (code 11000)
ConflictException "Ya compartiste este recurso con esa persona"
409
Muestra el error

Solo el propietario del recurso puede compartirlo. El acceso se resuelve con la cadena de ancestros (para carpetas) y con las comparticiones directas (para archivos y carpetas).

El índice único { recurso, destinatario } impide compartir dos veces el mismo recurso con la misma persona. El error 11000 se traduce a 409 Conflict.

Se registra actividad COMPARTIR con el correo del destinatario y el nivel concedido, para que el propietario pueda auditar quién tiene acceso.

Papelera


Propietario
SPA Angular
JwtAuthGuard (RolesGuard)
PapeleraController
PapeleraService
AccesoService
CarpetasRepository (Mongoose)
ArchivosRepository (Mongoose)
ActividadesService
MongoDB (carpetas + archivos + actividades)
Pulsa "Enviar a papelera" sobre una carpeta
POST /api/papelera/carpetas/:id Authorization: Bearer <jwt>
Verifica JWT y rol
permite la petición
ParseObjectIdPipe1(:id)
enviarCarpeta(id, actual)
buscarActiva(id, actual.id)
findOne(carpetas, {_id, propietario, enPapelera:false})
carpeta | null
carpeta | null
alt
[carpeta no existe o no es del usuario]
exigirCarpeta(actual.id, id, 'PROPIETARIO')
ForbiddenException / NotFoundException
error
403 / 404
Muestra el error
[carpeta válida]
fecha = new Date() (una sola para todo el lote)
idsDescendientesActivas(carpeta.id)
find(carpetas, {ancestros: id, enPapelera:false}) select _id
subcarpetas
subcarpetas[]
enviarAPapeleraPorCarpetas([carpeta._id, ...subcarpetas], fecha)
updateMany(archivos, {carpeta: {$in: [...]}, enPapelera:false}, {$set: {enPapelera:true, fechaEliminacion: fecha, eliminadoPorCascada: true}})
archivos modificados
nArchivos
enviarDescendientesAPapelera(carpeta.id, fecha)
updateMany(carpetas, {ancestros: id, enPapelera:false}, {$set: {enPapelera:true, fechaEliminacion: fecha, eliminadaPorCascada: true}})
subcarpetas modificadas
nSubcarpetas
enviarAPapelera(carpeta.id, fecha)
findOneAndUpdate(carpetas, {_id: id, enPapelera:false}, {$set: {enPapelera:true, fechaEliminacion: fecha, eliminadaPorCascada: false}})
carpeta actualizada
carpeta
registrar(ELIMINAR, CARPETA, recurso, nombre, {subcarpetas, archivos})
insertOne(actividades)
carpeta en papelera
200 OK
Muestra confirmación (la carpeta desaparece del explorador)

El lote completo (carpeta + subcarpetas + archivos) comparte la MISMA fechaEliminacion. Esto permite restaurar exactamente lo que se eliminó junto (restaurarCarpeta usa esa fecha como criterio).

La carpeta raíz se marca con eliminadaPorCascada=false (es la "raíz" del lote). Las subcarpetas y archivos se marcan con true, y NO aparecen en el listado de la papelera (solo se ven las raíces).

Si una subcarpeta está vacía, igual se marca como eliminadaPorCascada=true. Al restaurar la raíz, se restaura el lote completo.